

ROS2 3일차 과제2 보고서, 김도훈, 2026406041

<목차>

1. 과제2 설명
2. 과제2 구현
3. 과제2 실행 결과

1.과제2 설명

과제2는 QT를 사용해서 교차로 시뮬레이터를 구현하는 것이다. QT는 오직 시각화 용도이고 신호등 상태를 관리하는 노드와 차량을 생성하고 이동시키는 노드에서 모든 로직을 처리 한 다음 최종 결과만 토픽을 통해 Qt에 전달한다. 신호등은 규칙에 맞게 점등하고 자동차는 신호 등에 불에 해당하는 동작을 하도록 구현 해야한다. 이번 과제는 각각 노드들의 유기적으로 잘 연결되어야 하는 과제이다.

2.과제2 구현

먼저 과제2를 구현하기 전에 각 파일들에 어떤 기능들을 구현할 것 인지 정하고 구현을 시작했다. 먼저 본 프로그램 파일은 traffic_light_node.cpp, vehicle_node.cpp, qnode.cpp, mainwindow.cpp. 이 파일들을 사용한다. 먼저 traffic_light_node.cpp 노드의 주된 기능은 특정 시간 간격 동안 신호등 불을 빨 -> 초 -> 노 -> 빨 규칙으로 점등 시켜야한다. 특정 시간은 yaml 파일에 파라미터로 사용될 것이다. vehicle_node.cpp 노드의 주된 기능은 신호등의 불빛에 맞게 자동차를 동작하게 하는 것이다. 이 노드에서 자동차의 속도를 관리하는데 이 속도도 신호등의 점등 시간과 같이 yaml 파일의 파라미터로 사용될 예정이다. qnode.cpp는 .cpp 노드들과 ui와 연결을 시켜주어야 한다. 신호등 노드에 publisher 한 것을 subscriber 해야하고 subscriber 한 데이터를 ui에게 전달해야한다. 신호등의 불을 점등하고 신호등의 불의 따라 자동차 동작들을 결정하는 모든 로직의 traffic_light_node.cpp와 vehicle_node.cpp에 있다.

신호등을 점등 시키는 기능은 traffic_light_node.cpp에서

```
void TrafficLightPublisher::change_light()
{
    auto msg = std_msgs::msg::String();

    if (state_ == 0)
    {
        msg.data = "RED";
        state_ = 1;
        set_timer();
    }

    else if (state_ == 1)
    {
        msg.data = "GREEN";
        state_ = 2;
```

```

set_timer();
}

else
{
    msg.data = "YELLOW";
    state_ = 0;
    set_timer();
}

publisher_ -> publish(msg);

set_timer();
}

```

위와 같은 함수로 순서대로 점들을 시킨도록 했고 mas.data로 정의된 데이터가 불을 바꿀 때 마다 publisher을 통해 qnode에 현재 점등된 불을 색깔을 전달한다. 그리고 set.timer()를 통해 불이 일정 시간 동안 켜지도록 하였다. 이 타이머 안에 들어갈 매개변수가 파라미터가 된다.

```

void TrafficLightPublisher::set_timer()
{
    time_ = this->get_parameter("time").as_double();
    timer_ =
        this->create_wall_timer(
            std::chrono::milliseconds(
                static_cast<int>(time_ * 1000)
            ),
            [this]()
            {
                timer_ -> cancel();
                change_light();
            }
        );
}

```

자동차는 움직임을 구현하는 노드는 vehicle_node.cpp에서 신호등 상태에 따라 속도를 결정하고 그 속도로 x좌표를 줄이는 방식이다. traffic_light_node.cpp 노드의 publisher을 통해 신호등의 상태를 알 수 있다. 이 vehicle_node.cpp노드에서는 이 정보를 subscribe하였고 이 과정을 통해 신호등 불에 맞게 자동차를 움직이게 구현했다.

```

void VehicleNode::update_vehicle()
{
    const double dt = 0.05;
    if (traffic_state_ == "GREEN")
    {
        current_speed_ = speed_;
    }
    else if (traffic_state_ == "YELLOW")
    {
        current_speed_ -= speed_ * dt;
        if (current_speed_ < 0.0)
        {
            current_speed_ = 0.0;
        }
    }
}

```

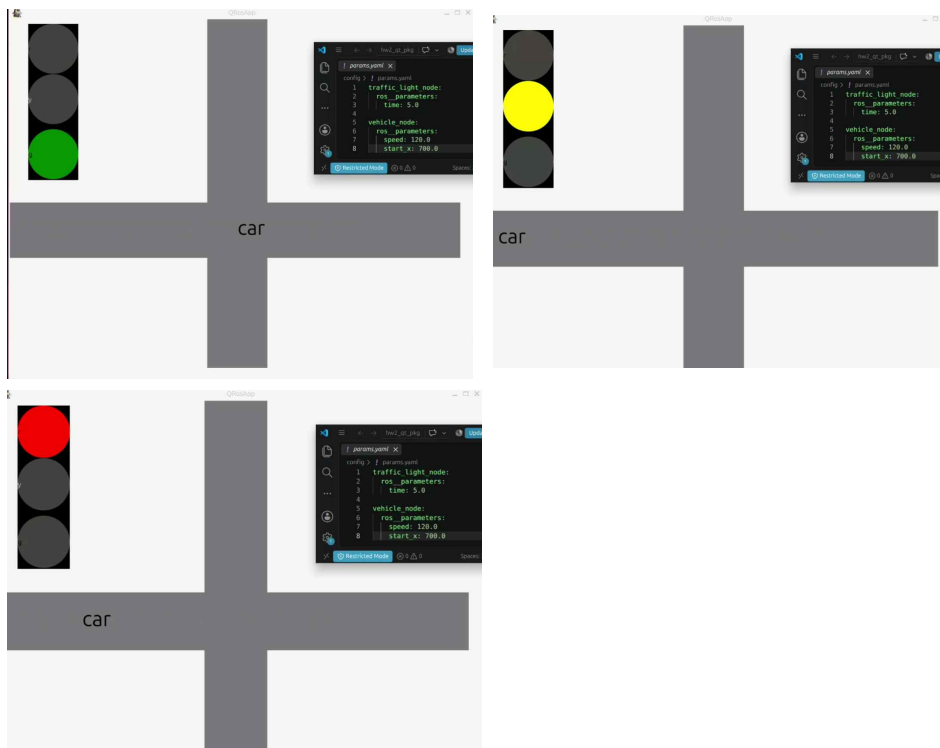
```
}
.....
```

`const double dt = 0.05;`는 시간 간격이다. 자동차가 움직일 때 50ms의 간격을 두고 움직이도록 하였다. 신호등 불이 초록색일 일 때는 파라미터로 사용되는 `speed_` 값이 유지되면서 자동차가 x축에서 움직이게 된다. 그리고 노란색 불일 때는 감속을 하다가 정지해야 하는데 이것을 구현하기 위해 `current_speed_ -= speed_ * dt;` 시간이 증가할수록 현재의 속도(움직이는 거리)가 감소하도록 하였고 속도가 0보다 작아면 정지 하도록 하여 감속/정지를 구현했다. 빨간불은 단순히 정지이기 때문에 속도값에 0을 주었다. 최종 로직은 다음과 같다.

1. `traffic_light_node.cpp` -> `msg.data = "GREEN"` publish
2. `traffic_light` 저등
3. `vehicle_node.cpp` -> `msg->data == "GREEN"`
4. `traffic_state_ = "GREEN"` -> 자동차 이동

그리고 본 프로젝트에서는 신호등 노드, 자동차 노드, Qt GUI 노드를 각각 실행해야 한다. 각 노드를 터미널에서 따로 실행하면 실행 과정이 번거롭기 때문에 `launch` 파일을 사용하였다.

3. 과제2 실행 결과



1. 실행 결과

점등 시간 간격 5초, 자동차의 속도 120, 자동차의 시작 위치 700

2. 실행 결과

점등 시간 간격 2초, 자동차의 속도 100, 자동차 시작 위치 700

